

Jobs & CronJobs

Batch and Scheduled Workloads

Intermediate Kubernetes — Batch Workloads (companion to Module 2)

Objectives

- Contrast **run-to-completion** workloads with long-running Deployments
- Run a **Job** with retries (`backoffLimit`) and completion tracking
- Fan work out with **completions** and **parallelism**
- Schedule recurring work with a **CronJob**
- Trigger a CronJob **on demand** and **suspend** it
- Auto-clean finished Jobs with `ttlSecondsAfterFinished`



Run a Pod to completion — with retries

Deployment vs. Job

Deployment

- Keeps Pods **running** indefinitely
- Restarts them on exit
(`restartPolicy: Always`)
- Success = "still serving"

Job

- Runs Pods **to completion**, then stops
- `restartPolicy`: Never or OnFailure
- Success = "the task finished"

Reach for a Job whenever the work has a natural *end*: a migration, a report, a backup, a one-off script.

Job Mechanics

Key fields

- `backoffLimit` — retries before the Job fails
- `completions` — successes required
- `parallelism` — Pods running at once
- `activeDeadlineSeconds` — wall-clock cap
- `ttlSecondsAfterFinished` — auto-cleanup

Status you'll watch

- `COMPLETIONS` column (e.g. 1/1, 6/6)
- Complete vs Failed condition
- `.status.succeeded` / `.status.failed`

A failed Pod is retried up to `backoffLimit`; exceed it and the whole Job is marked Failed.

Parallel Jobs

completions × parallelism

completions: 6 + parallelism: 3 → six units of work run **three at a time** until all six succeed.

- **Fixed count** — set completions for a known number of items
- **Work queue** — omit completions; Pods pull from a queue until it's empty
- **Indexed** (completionMode: Indexed) — each Pod gets a unique index for sharded work

This is the batch fan-out pattern: split work into N units, process P concurrently.



CronJobs

Create Jobs on a schedule



CronJob Anatomy

Schedule

Standard 5-field cron: `"*/1 * * * *"` (every minute), `"0 2 * * *"` (2am daily).

A CronJob wraps a `jobTemplate` — every fire creates a fresh Job.

Controls

- `concurrencyPolicy`: Allow / Forbid / Replace
- `startingDeadlineSeconds` — miss window handling
- `successfulJobsHistoryLimit` / `failedJobsHistoryLimit`
- `suspend`: true — pause scheduling

Forbid skips a run if the previous one is still going — essential for jobs that must not overlap.



Running & Pausing on Demand

Test a CronJob without waiting for the clock:

```
kubectl create job --from=cronjob/<name> run-now
```

- **Suspend** — `kubectl patch cronjob <name> -p '{"spec":{"suspend":true}}'` stops new Jobs
- **History** — finished Jobs linger per the history limits, so you can read their logs
- **Cleanup** — `ttlSecondsAfterFinished` removes completed Jobs automatically

Always test a CronJob's Job template with `--from` before trusting the schedule.

Choosing a Workload

Work	Use
Long-running server / API	Deployment
One-off task that finishes (migration, report)	Job
Recurring task (nightly backup, cleanup)	CronJob
Fan-out batch (N items, P at a time)	Job with completions/parallelism

If it should ever "finish," it's a Job — not a Deployment with a sleep loop.

Lab: Jobs and CronJobs

Hands-on: batch and scheduled workloads

- Run a **Job** to completion and read its results
- Fan out a **parallel Job** (`completions: 6, parallelism: 3`)
- Schedule a **CronJob** and inspect the Jobs it creates
- Trigger it with `--from=cronjob` and **suspend** the schedule

Duration: ~25-35 minutes

Key Takeaways

- A **Job** runs a Pod *to completion* with retries (`backoffLimit`); use `restartPolicy: Never/OnFailure`.
- **completions + parallelism** fan work out; work-queue and indexed modes handle dynamic and sharded batches.
- A **CronJob** creates Jobs on a cron schedule; `concurrencyPolicy` controls overlap.
- `kubectl create job --from=cronjob/<name>` runs it now; `suspend: true` pauses it.
- `ttlSecondsAfterFinished` keeps finished Jobs from piling up.